
Software Requirements Specification

for

FIFA World Cup Data Analytics & Visualization Platform

Version <1.0>

Prepared by

Group Name: Team NO.1

Saketh Krishna Rao	Se23uari013	se23uari013@mahindrauniversity.edu.in
Kushal	Se23uari073	se23uari073@mahindrauniversity.edu.in
Adrija Bandi	Se23uari004	se23uari004@mahindrauniversity.edu.in
Bhogi Charan Tej	Se23ucse240	se23ucse240@mahindrauniversity.edu.in
Nithya Rao	Se23uari125	se23uari125@mahindrauniversity.edu.in
Meka Lakshmi Amrutha	Se23ucse110	se23ucse110@mahindrauniversity.edu.in
Sameehan Sriharsha	Se23uari069	se23uari069@mahindrauniversity.edu.in

Instructor:

Avinash Chauhan

Course:

Software Engineering

Lab Section:

First Floor Lab

Teaching Assistant:

Murali Krishna Bukkasamudram

Date:

10-03-2025

Contents

CONTENTS	II
REVISIONS	III
1 INTRODUCTION	1
1.1 DOCUMENT PURPOSE	1
1.2 PRODUCT SCOPE	1
1.3 INTENDED AUDIENCE AND DOCUMENT OVERVIEW.....	1
1.4 DEFINITIONS, ACRONYMS AND ABBREVIATIONS.....	2
1.5 DOCUMENT CONVENTIONS	2
1.6 REFERENCES AND ACKNOWLEDGMENTS.....	2
2 OVERALL DESCRIPTION	3
2.1 PRODUCT OVERVIEW	3
2.2 PRODUCT FUNCTIONALITY	4
2.3 DESIGN AND IMPLEMENTATION CONSTRAINTS	4
2.4 ASSUMPTIONS AND DEPENDENCIES	4
3 SPECIFIC REQUIREMENTS	5
3.1 EXTERNAL INTERFACE REQUIREMENTS	5
3.2 FUNCTIONAL REQUIREMENTS	6
3.3 USE CASE MODEL.....	8
4 OTHER NON-FUNCTIONAL REQUIREMENTS	11
4.1 PERFORMANCE REQUIREMENTS.....	11
4.2 SAFETY AND SECURITY REQUIREMENTS.....	11
4.3 SOFTWARE QUALITY ATTRIBUTES	11
5 OTHER REQUIREMENTS	12
APPENDIX A – DATA DICTIONARY	12
APPENDIX B - GROUP LOG	12

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
1.0	Saketh krishna, Kushal, Adrija Bandi, Charan Tej, Nithya Rao, Sriharsha, Amrutha	Description of Version 1.0: Software Requirements Specification (SRS) for the FIFA World Cup Data Analytics & Visualization Platform	08/03/2026

1 Introduction

1.1 Document Purpose

This Software Requirements Specification (SRS) document describes the requirements for the **FIFA World Cup Data Analytics and Visualization Platform**. The system is designed to collect, process, analyze, and visualize historical FIFA World Cup data in an interactive web-based dashboard.

The purpose of this document is to provide a detailed description of system requirements including functional requirements, system interfaces, performance expectations, and constraints. It serves as a reference for developers, project stakeholders, instructors, and testers involved in the development of the system.

The project will follow the Incremental Software Development Model, allowing system functionality to be implemented and tested in stages.

1.2 Product Scope

The FIFA World Cup Data Analytics Platform is a web-based application that enables users to explore and analyze historical football match data through visual dashboards and analytical insights.

The system processes raw FIFA datasets, cleans and validates them, computes analytics such as win rates and goal differences, and presents them using interactive visualizations.

Key benefits Include:

- 1. Interactive exploration of historical FIFA World Cup data.*
- 2. Automated analytics and statistical insights.*
- 3. High-performance REST API for retrieving football statistics.*
- 4. Data-driven visual dashboards for analysts and researchers.*

The platform supports analysts, fans, and researchers who want deeper insights into FIFA World Cup performance trends.

1.3 Intended Audience and Document Overview

This document is intended for:

- 1. Developers** – to understand system functionality and architecture
- 2. Project instructors** – to evaluate project design and requirements
- 3. Testers** – to validate system functionality
- 4. Stakeholders** – to understand system objectives

The document is structured as follows:

- Section 1 introduces the project
- Section 2 describes the system overview
- Section 3 provides detailed system requirements
- Section 4 outlines non-functional requirements
- Section 5 includes additional requirements and constraints

1.4 Definitions, Acronyms and Abbreviations

Term	Definition
API	Application Programming Interface
CSV	Comma Separated Values
EDA	Exploratory Data Analysis
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
REST	Representational State Transfer
SQL	Structured Query Language
TTL	Time To Live
UI	User Interface

1.5 Document Conventions

This document follows IEEE SRS documentation standards.

Conventions used:

- **Section numbering** follows IEEE format
- **Bold text** represents system components
- **Bullet lists** represent system features
- **Functional requirements** follow the format F1, F2, etc.

1.6 References and Acknowledgments

References used for this system include:

- *Kaggle FIFA World Cup datasets*
- *FIFA official statistics*
- *FastAPI documentation*
- *PostgreSQL documentation*
- *Redis documentation*
- *ReactJS and D3.js documentation*

2 Overall Description

2.1 Product Overview

The FIFA World Cup Data Analytics Platform is a full-stack data analytics system that processes historical FIFA datasets and presents analytical insights through a web dashboard.

The system consists of five primary layers:

1. Data Acquisition and Cleaning Layer
2. Analytics and Insights Layer
3. Backend API and Database Layer
4. Frontend Visualization Layer
5. Performance Testing and Documentation Layer

Data flows from raw datasets through cleaning pipelines and analytics modules before being stored in databases and accessed through APIs.

Users interact with the system through a web dashboard that displays interactive charts and statistics.

Figure 1 shows the high-level architecture of the FIFA World Cup Data Analytics & Visualization Platform. The system consists of multiple layers including data acquisition, analytics processing, backend API services, caching mechanisms, and a frontend visualization dashboard.

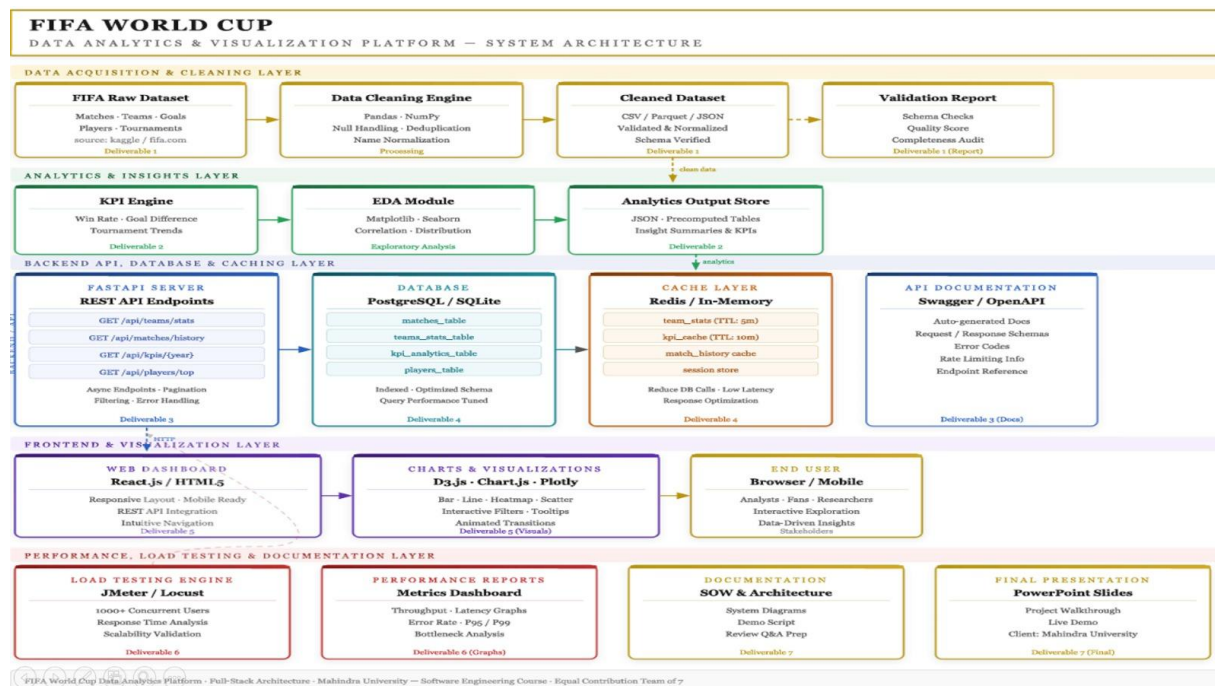


Figure 1: System Architecture of the FIFA World Cup Data Analytics & Visualization Platform

2.1.1 Incremental Development Approach

- The FIFA World Cup Data Analytics & Visualization Platform will be developed using the Incremental Software Development Model.
- The system will be delivered in multiple increments where each increment implements a subset of the system functionality.
- Each increment will be independently developed, tested, and integrated with previously completed increments.
- This approach enables **early delivery of working modules** and continuous system improvement through iterative development.

2.2 Product Functionality

Major functions of the system include:

- Import FIFA World Cup datasets
- Clean and validate raw data
- Generate analytics metrics such as win rates and goal differences
- Perform exploratory data analysis
- Store processed data in databases
- Provide REST APIs for retrieving analytics
- Cache frequently requested statistics
- Visualize data through charts and dashboards
- Support interactive exploration of football statistics

These functionalities will be implemented progressively through multiple increments of the system development process.

2.3 Design and Implementation Constraints

The system must follow several technical constraints:

- Backend must be implemented using **FastAPI**
- Database must use **PostgreSQL or SQLite**

- Caching must use **Redis**
- Data processing must use **Python with Pandas and NumPy**
- Visualization must use **D3.js, Chart.js, or Plotly**
- Frontend must use **React.js**
- API documentation must follow **OpenAPI / Swagger standards**
- System design follows **UML modeling principles**

2.4 Assumptions and Dependencies

The system assumes the following:

- *FIFA datasets are available from Kaggle or FIFA sources*
- *Users access the system via modern web browsers*
- *The database server is available and operational*
- *Internet connectivity is stable*
- *APIs are accessible via HTTP*

Dependencies include:

- *Python libraries (Pandas, NumPy, Matplotlib)*
- *PostgreSQL database*
- *Redis caching system*
- *ReactJS frontend framework*

3 Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The system provides a **web dashboard interface** accessible through a browser.

Users interact with the dashboard through:

- Interactive charts
- Filtering options
- Statistical summaries
- Tooltips and animations

The interface is responsive and supports both desktop and mobile devices.

3.1.2 Hardware Interfaces

The system interacts with the following hardware components:

- Web servers hosting the backend API
- Database servers running PostgreSQL

- User devices such as laptops, tablets, and smartphones

3.1.3 Software Interfaces

The system integrates with several software components:

- **FastAPI** for REST API services
- **PostgreSQL** for structured data storage
- **Redis** for caching
- **ReactJS** frontend framework
- **Chart.js** / **D3.js** visualization libraries

3.2 Functional Requirements

3.2.1 F1: Data Import

The system shall import raw FIFA World Cup datasets from external sources.

3.2.2 F2: Data Cleaning

The system shall clean datasets by removing duplicates, handling missing values, and normalizing names.

3.2.3 F3: Data Validation

The system shall validate dataset schema and generate validation reports.

3.2.4 F4: KPI Generation

The system shall compute analytics metrics including:

- Team win rate
- Goal difference
- Tournament performance trends

3.2.5 F5: API Access

The system shall expose REST API endpoints to retrieve football statistics.

Example endpoints:

```
GET /api/teams/stats  
GET /api/matches/history  
GET /api/kpis/{year}  
GET /api/players/top
```

3.2.6 F6: Data Storage

The system shall store processed data in a relational database.

3.2.7 F7: Caching

The system shall cache frequently accessed data using Redis.

3.2.8 F8: Data Visualization

The system shall display analytics results through interactive charts.

Requirements Traceability Table:

Requirement ID	Description	Use Case
F1	Import FIFA dataset	U2
F2	Clean dataset	U2
F3	Validate dataset	U3
F4	Generate KPIs	U3
F5	Provide API endpoints	U1, U2
F6	Store processed data	U1
F7	Cache frequently accessed data	U3
F8	Visualize analytics	U1, U3

Increment Requirement Mapping

Increment	Features Implemented
Increment 1	F1, F2, F3, F6
Increment 2	F4, F5, F7
Increment 3	F8, U1, U2, U3

3.3 `Use Case Model

- **Actors**
- **Analyst** – Uses the system to analyze football statistics.
- **Fan/User** – Explores match history and team performance.
- **Researcher** – Studies trends in World Cup tournaments.

3.3.1 Use Case 1 - View Team Statistics

Author: Charan Tej

Purpose:

Allows users to view statistical performance of football teams including win rate, goal difference, and historical performance trends.

Requirements Traceability

F5 – API Access

F6 – Data Storage

F8 – Data Visualization

ssPriority

High

Preconditions

- Team statistics data must exist in the database.
- Backend API server must be running.

Postconditions

- Team statistics are displayed to the user through the dashboard interface.
- User can analyze past tournament matches.

Actors

- Analyst
- Fan/User
- Researcher

Extends

None

Flow of Events

Basic Flow

1. User opens the web dashboard.
2. User selects **Team Statistics** from the dashboard.
3. Frontend sends API request to /api/teams/stats.
4. Backend retrieves team statistics from the database.
5. API returns team performance data in JSON format.
6. Frontend displays statistics such as win rate, goal difference, and tournament performance.

Alternative Flow

A1 – Filter by Team

1. User selects a specific team.
2. System sends request with team parameter.
3. Backend retrieves statistics for the selected team.
4. Filtered statistics are displayed.

Exceptions**E1 – No Data Found**

1. System cannot find statistics for selected team.
2. System displays message:

“No statistics available for the selected team.”

Includes

None

Notes / Issues

Pagination may be required for large datasets.

3.3.2 Use Case 2 - View Match History

Author: Charan Tej

Purpose:

Allows users to explore historical FIFA World Cup match data, including teams, goals scored, and match results.

Requirements Traceability

F1 – Data Import
F2 – Data Cleaning
F5 – API Access
F6 – Data Storage
F8 – Data Visualization

Priority

High

Preconditions

- Match history data must exist in the database.
- Backend API server must be running.

Postconditions

- Match history results are displayed in tabular or graphical format.
- User can analyze past tournament matches.

Actors

- Analyst
- Fan/User
- Researcher

Extends

None

Flow of Events**Basic Flow**

1. User opens the web dashboard.
 2. User selects **Match History**.
 3. Frontend sends API request to /api/matches/history.
 4. Backend retrieves match records from database.
 5. API returns match data in JSON format.
 6. Frontend displays match history in charts or tables.
-

Alternative Flow**A1 – Filter Matches by Year**

1. User selects a specific tournament year.
2. API request includes filter parameter.

3. Backend retrieves matches only for that year.
 4. Filtered results are displayed.
-

Exceptions**E1 – No Data Found**

1. User selects a year with no match data.
 2. System displays message:
“No match data available for selected year.”
-

Includes

None

Notes / Issues

- Pagination may be required for large datasets.

3.3.3 Use Case 3 – View KPI Analytics Dashboard

Author: Charan Tej

Purpose

Allows users to view analytics insights such as win rate, goal differences, and tournament trends through interactive dashboards.

Requirements Traceability

F3 – Data Validation

F4 – KPI Generation

F5 – API Access

F8 – Data Visualization

Priority

High

Preconditions

- Cleaned dataset must be available.
- Analytics metrics must be computed.

Postconditions

- KPI analytics are displayed through interactive charts.
- Users can interpret statistical insights.

Actors

- Analyst
- Researcher
- Fan/User

Extends

None

Flow of Events**Basic Flow**

1. User opens the **Analytics Dashboard**.
 2. Frontend requests KPI data from `/api/kpis/{year}`.
 3. Backend retrieves KPI metrics.
 4. API returns analytics data.
 5. Frontend renders visualizations using D3.js or Chart.js.
 6. User interacts with charts and tooltips.
-

Alternative Flow**A1 – Cached KPI Data**

1. System retrieves KPI metrics from Redis cache.
2. Data is returned instantly to frontend.

Exceptions

E1 – Analytics Data Not Generated

1. KPI data is missing.
 2. System returns error message.
-

IncludesU1

Notes / Issues

- KPI calculations must be optimized to avoid slow queries.

4 Other Non-functional Requirements

4.1 Performance Requirements

P1. API response time shall be **less than 500 ms** for cached queries.

P2. The system shall support **1000 concurrent users** during load testing.

P3. Data processing pipeline shall complete dataset cleaning within **60 seconds**.

P4. The dashboard analytics results shall be displayed to the user within 2 seconds under normal operating conditions.

4.2 Safety and Security Requirements

The system shall include the following security mechanisms:

- API rate limiting
- Input validation
- Protection against SQL injection
- Secure API documentation using Swagger
- HTTPS communication for data transfer

4.3 Software Quality Attributes

4.3.1 Reliability

The system shall ensure reliable data processing and prevent corruption of datasets.

4.3.2 Maintainability

The system architecture is modular to allow future extensions such as:

- *Additional sports analytics*
- *New visualization modules*

- New dataset integration

4.3.3 Usability

The dashboard interface shall provide intuitive navigation and interactive charts.

5 Other Requirements

Database schema must include the following tables:

- *matches_table*
- *teams_stats_table*
- *players_table*
- *kpi_analytics_table*

The system must also support exporting analytics data in JSON or CSV format.

Appendix A – Data Dictionary

Variable	Description
<i>team_id</i>	<i>Unique identifier for each football team</i>
<i>match_date</i>	<i>Date when the match was played</i>
<i>stadium</i>	<i>Stadium where the match occurred</i>
<i>winner</i>	<i>Winning team</i>

Appendix B - Group Log

Group meetings include:

- *System architecture design*
- *Dataset selection*
- *Backend API design*
- *Dashboard development*
- *Performance testing*